CYB333 Log Monitor

Security Log Parsing Project

Repository: https://github.com/Kai00-pen/CYB333_Log_Monitor

Project overview

This project uses Python to read a text file of security events and organize each entry so it is easier to review. The sample file records login attempts, an authentication failure, and a blocked network connection. Rather than showing the information as one long line, the program labels the time, event, username, IP address, and result for each entry.

At this stage, the project is a log parser. It does not monitor a live system, detect attacks automatically, or send alerts. Those features could be added later, using the information the parser already prepares.

Project files

log_parser.py — The Python program that reads, separates, and displays log entries.

sample_logs.txt — Seven sample security events used to test the program.

README — Instructions and explanation of the project.

How the sample logs are organized

Each line uses the same five-part format, with a vertical bar between fields:

```
2026-10-05 10:17:03 | LOGIN | admin | 192.168.1.25 | FAILED
```

The date and time tell us when the event occurred. LOGIN is the type of event. admin is the account involved. The IP address identifies the network address recorded with the attempt, and FAILED tells us the login was unsuccessful. This consistent layout is what allows the Python program to split the entry into useful pieces.

Set up and run the project

1. Install Python from python.org if it is not already installed. In a terminal, run python --version to check that it works. Some systems use python3 instead.

2. Download the repository from GitHub using Code > Download ZIP, then extract the folder. You can also clone it with Git:

```
git clone https://github.com/Kai00-pen/CYB333_Log_Monitor.git
```

3. Open the project folder in Visual Studio Code. Confirm that log_parser.py and sample_logs.txt are both present.

4. Open Terminal > New Terminal in VS Code. Make sure the terminal is in the project folder, then run:

```
python log_parser.py
```

If that command does not work because of your Python installation, try python3 log_parser.py (or py log_parser.py on Windows).

5. The terminal should print “=== Processed Security Logs ===” followed by seven events. Each event will have its own Time, Event, Username, IP Address, and Status lines.

How the Python script works

1. Reading the file: read_logs()

```
def read_logs(filename):
    logs = []

    try:
        with open(filename, "r") as file:
            for line in file:
                line = line.strip()
                if line:
                    logs.append(line)
    except FileNotFoundError:
        print(f"Error: {filename} was not found.")

    return logs
```

The function starts with an empty list named logs. It opens the requested file in read mode, goes through it one line at a time, and removes extra spaces and line breaks using strip(). Empty lines are skipped. Every remaining line is added to the list with append(). If the file is missing, the function prints an error instead of stopping with an uncaught FileNotFoundError. Finally, it returns the list of log entries.

2. Separating the fields: parse_logs()

```
def parse_logs(logs):
    parsed_logs = []

    for log in logs:
        parts = log.split(" | ")
        if len(parts) == 5:
            log_entry = {
                "timestamp": parts[0],
                "event": parts[1],
                "username": parts[2],
```

```
                "ip_address": parts[3],
                "status": parts[4]
            }
            parsed_logs.append(log_entry)

    return parsed_logs
```

The split(" | ") instruction divides one log line wherever the separator appears. Python then checks that it found exactly five pieces. If so, the values are stored in a dictionary, which is simply a set of named fields. For example, parts[2] becomes the username. Entries with the wrong number of fields are skipped. The code does not check whether an IP address or timestamp is valid, so five fields alone do not guarantee a correct entry.

3. Printing the results: display_logs()

```
def display_logs(parsed_logs):
    print("=== Processed Security Logs ===")

    for log in parsed_logs:
        print(f"Time: {log['timestamp']}")
        print(f"Event: {log['event']}")
        print(f"Username: {log['username']}")
        print(f"IP Address: {log['ip_address']}")
        print(f"Status: {log['status']}")
        print("-" * 40)
```

This function loops through the processed entries and prints each field on a separate line. The f-strings insert the stored values into the labels. The row of hyphens separates one event from the next, making the terminal output easier to follow.

4. Putting the steps together: main()

```
def main():
    filename = "sample_logs.txt"
    logs = read_logs(filename)
    parsed_logs = parse_logs(logs)
    display_logs(parsed_logs)
    return parsed_logs

if __name__ == "__main__":
    main()
```

The main function runs the steps in order: read the file, parse the entries, and print the results. It also returns the processed list, which could be reused by other Python code later. The final if statement runs main() when this script is launched directly, rather than imported into another script.

What the sample results show

The seven supplied entries include two successful logins, four failed events, and one blocked connection. Three of the failed LOGIN events involve the admin account from 192.168.1.25, at 10:17:03, 10:17:15, and 10:17:29 on October 5, 2026. That is three failures within 26 seconds. This is worth reviewing, but the sample does not establish that an attack occurred.

Another failed event is marked AUTH for jdoe, and the last entry records a blocked CONNECTION. The program prints all of these events, but it does not count them or raise warnings by itself.

Simple tests to try

Test 1: Run the script without changing anything. Confirm that seven events appear.

Test 2: Add a correctly formatted entry to the end of sample_logs.txt, save the file, and run the script again. The new entry should appear in the terminal.

```
2026-10-05 10:30:00 | LOGIN | testuser | 192.168.1.50 | FAILED
```

Test 3: Temporarily rename sample_logs.txt and run the script. It should print an error saying the file was not found, then display the heading with no entries. Restore the filename afterward.

Test 4: Add an entry with fewer than five fields. The program should skip that entry because it does not match the required structure.

Common problems

If Python is not recognized, check the installation and try python3 or py. If the log file cannot be found, confirm that the terminal is running in the folder containing sample_logs.txt. If some entries are missing, inspect the separators and make sure each line has five fields. Remember to save the text file before rerunning the script.

Possible next steps

The next logical improvement would be a function that counts failed logins by account or IP address. After that, the project could compare counts within a set time period and display a warning when a threshold is reached. It would also be useful to report malformed lines rather than silently skip them. These are future ideas, not features in the current code.

Closing notes

This project is a practical starting point for working with security logs in Python. It demonstrates file handling, loops, lists, dictionaries, conditions, functions, and basic error handling in one

script. Most importantly, it turns a plain text log into information that can be reviewed more easily and later used for security analysis.